Architecting a Local-First Observability Primitive: A Technical Specification for the Prompt Lab v1 Run Object Schema

The landscape of prompt engineering has transitioned from isolated experimentation toward a complex lifecycle involving iterative versioning, multi-turn tracing, and rigorous performance evaluation. As the engineering community increasingly adopts local-first paradigms to mitigate the latency, cost, and privacy concerns inherent in cloud-centralized observability, the design of the "Run Object" becomes the foundational primitive for all downstream analysis. In a browser-extension-first architecture, this object must not only encapsulate the intent and response of a large language model (LLM) but also preserve the hierarchical context of the execution while navigating the technical constraints of ephemeral background processes and browser-based storage engines. This report provides an exhaustive synthesis of industry standards and architectural trade-offs to define the v1 Run Object schema for Prompt Lab.

Evolutionary Taxonomy of LLM Observability Schemas

The current state of the art in LLM observability is defined by four primary philosophical approaches: the experimental tracking lineage of Weights & Biases, the framework-native tracing of LangSmith, the middleware-centric blueprinting of PromptLayer, and the proxy-based metadata enrichment of Helicone. A rigorous comparison of these schemas reveals the necessary components for a unified local-first representation.

Weights & Biases: The Transition to Call-Based Tracing

Weights & Biases (W&B) initially established its dominance through experiment tracking for traditional machine learning, but its "Weave" and "Prompts" integrations represent a pivot toward functional observability. The W&B Call object is structured around the concept of an operation as a traceable unit.[1] It utilizes unique identifiers (UUIDs) for calls, traces, and parent-child relationships, ensuring that every inference step can be reconstructed within a larger DAG (Directed Acyclic Graph).

The W&B schema makes a critical distinction between attributes and inputs. Attributes are defined as user-supplied metadata that are frozen at the moment of execution, providing a static context for the run.[1] This is conceptually distinct from inputs and outputs, which are dynamic and subject to the execution flow. Post-execution data is captured in a summary property, a flexible map that records costs, latencies, and final status (SUCCESS, ERROR, RUNNING, or DESCENDANT_ERROR).[1] This tripartite structure—static context, dynamic payload, and post-execution summary—is highly efficient for local-first tools where certain environmental variables (like the extension version or browser context) are known prior to the

network request to the LLM provider.

LangSmith: Hierarchical Persistence and Dotted Order

LangSmith, developed by the LangChain team, offers perhaps the most sophisticated schema for handling the complex hierarchies of agentic workflows. A LangSmith Run represents either a trace (root run) or a child run (span).[2] To manage the relationship between these entities without incurring the performance costs of recursive joins in a database, LangSmith employs a dotted_order field.[3] This string encodes the timestamp and UUID of the current run, prefixed by its parent's dotted order, creating a lexicographically sortable path that represents the entire ancestry of the execution.[3]

For a local-first browser extension, where storage engines like IndexedDB may lack the relational depth of a traditional SQL server, the dotted_order mechanism is an essential architectural pattern. It allows the UI to reconstruct tree views of complex chains with a single range query. Furthermore, LangSmith's schema includes explicit inputs_schema and outputs_schema fields, facilitating the validation of structured data—a necessity when logging the outputs of tools or functions.[2] The inclusion of commit_hash and prompt_id links the execution of a run directly to the version control system of the prompt registry, ensuring that any single run can be reproduced by pulling the specific template version.[5]

PromptLayer: The Blueprint and Metadata Enrichment

PromptLayer focuses on the prompt as a managed asset, and its schema reflects a commitment to structural integrity through the "Prompt Blueprint" format. Unlike simpler systems that store prompts as raw strings, PromptLayer requires message content to be an array of content blocks.[7] Each block has a specific type, such as text, image_url, or the more recent thinking block used for reasoning models like Claude 3.7 and OpenAI o1.[7]

The PromptLayer schema is particularly robust in its error tracking and status reporting. It introduces a WARNING status for requests that succeeded but encountered non-fatal issues, such as retries or missing variables.[8] It also categorizes errors into specific types like PROVIDER_TIMEOUT, PROVIDER_RATE_LIMIT, and TEMPLATE_RENDER_ERROR, which provides a much higher resolution of diagnostic data than a simple boolean success flag.[8] For Prompt Lab, incorporating this level of granularity is vital for identifying whether a failure originated in the extension's local logic or the provider's API.

Helicone: Proxy-Based Metadata and Performance Profiling

Helicone's schema is unique because it is designed to be populated primarily through HTTP headers. It uses a "Prompt ID" system to track versions and facilitates A/B testing by allowing users to override models and parameters at the gateway level.[9] One of its most distinctive contributions is the explicit tracking of Time to First Token (TTFT) through its manual logger and

streaming helpers.[11]

In Helicone's view, a run is part of a session, and it uses Helicone-Session-Id and Helicone-Session-Path to represent multi-turn conversations.[9] This session-path approach is a simplified version of LangSmith's dotted order, but it emphasizes the user's conversational journey. Helicone also incorporates a _type field in its custom logs, allowing it to unify LLM calls with vector database searches and tool executions into a single observability stream.[11] This model-agnostic and operation-agnostic design is a key requirement for Prompt Lab, which aims to be the central observability hub for all extension-based AI activities.

Vendor Schema Comparison Matrix

Attribute Category	Weights & Biases (Weave)	LangSmith	PromptLayer	Helicone
Primary Identity	id (UUID) [1]	id (UUID v7) [3]	request_id (pl_id) [14]	Helicone-Auth / Header-based [9]
Trace Hierarchy	trace_id, parent_id [1]	trace_id, dotted_order [3]	Spans Bulk API [8]	Session-Id, Session-Path [9]
Run Classification	op_name [1]	run_type (llm, tool, etc) [2]	api_type [7]	_type (custom log) [12]
Payload Structure	inputs, output (Dict) [1]	inputs, outputs (KVMap) [3]	input, output (Blueprint) [7]	input, response (JSON) [15]
Status Resolution	SUCCESS, ERROR, RUNNING [1]	OK, ERROR, UNSET [16]	SUCCESS, WARNING, ERROR [8]	Success/Error boolean [12]
Performance Data	latency_ms [1]	start_time, end_time [3]	request_start_time [7]	TTFT, latency [11]
Economic Data	costs [1]	usage_metadata [2]	price, tokens [7]	cost, token_usage [17]

Versioning Link	op_name (Reference) [1]	prompt_id, commit_hash [5]	prompt_id, prompt_version [7]	Helicone-Prompt-Id [9]

Browser Storage Architecture Findings: Local-First Persistence

The architecture of a local-first prompt engineering tool must reconcile the browser's storage limitations with the high-throughput requirements of LLM logging. A typical run involves complex JSON objects, metadata tags, and potentially large binary blobs (for multimodal inputs). The transition to Manifest V3 (MV3) in modern browsers has further complicated this by removing persistent background scripts in favor of ephemeral service workers.[18]

IndexedDB and Dexie.js: The Native Object Store

IndexedDB is the only web API capable of storing significant amounts of structured data persistently.[20] Unlike localStorage, which is limited to roughly 5 MB of string data and is prone to clearing during storage pressure, IndexedDB can utilize up to 80% of total disk space in Chromium-based browsers.[21] For a tool like Prompt Lab, the native support for JSON objects is a significant advantage, as it avoids the CPU overhead of constant stringification and parsing.[22]

Dexie.js serves as a critical abstraction layer over IndexedDB, addressing its notoriously verbose and complex API.[23] Dexie provides robust indexing, allowing for high-performance searches across tags, model names, or timestamps.[23] However, research indicates that IndexedDB write throughput is roughly 10 times slower than localStorage.[22] Furthermore, write performance can degrade significantly when moving operations from the main thread to a web worker, a common pattern in extension development.[25] In tests involving 100,000 small objects, main-thread writes completed in approximately 2.9 seconds, while worker-thread writes took over 21 seconds in certain Chrome versions.[25] This suggests that while IndexedDB is the most viable persistent store, Prompt Lab should use Dexie’s bulkAdd and bulkPut methods to minimize transaction overhead.[25]

SQLite WASM and the Origin Private File System (OPFS)

The advent of SQLite WASM, backed by the Origin Private File System (OPFS), offers a powerful alternative for developers requiring the full capabilities of a relational database.[26] OPFS provides a private, high-performance file system hidden from the user, allowing SQLite to operate at speeds comparable to native database engines.[26]

SQLite’s primary advantage is its support for complex JOINs, aggregates, and full-text search, which are difficult to implement efficiently in IndexedDB.[21] For analytical use cases—such as "find the average latency of all gpt-4o runs where the 'relevancy' score is > 80"—SQL is

significantly more expressive. However, SQLite WASM comes with substantial costs. The WASM binary itself is large, increasing the extension's footprint, and initializing the database can take several hundred milliseconds.[22] Furthermore, OPFS is not supported in Safari's incognito mode, necessitating a fallback to IndexedDB or IDBBatchAtomicVFS.[28] For a v1 implementation, the complexity of managing SQLite within an MV3 service worker environment—which may require an "offscreen document" to keep the database connection alive—outweighs the query benefits for most initial use cases.[29]

Comparative Storage Performance and Capacity Matrix

Metric	IndexedDB (Dexie.js)	SQLite WASM (OPFS)	localStorage
Write Latency	~0.17ms per record [22]	~3.0ms (single) / High (bulk) [22]	~0.017ms per record [22]
Storage Capacity	~80% of disk space [21]	Same as IndexedDB [22]	5-10 MB (per origin) [22]
Query Flexibility	Range/Index (Simple) [23]	SQL JOINs / Aggregates [21]	Key-only [26]
JSON Support	Native Object Storage [22]	JSON in Text Column (v3.38+) [22]	Stringified only [22]
Reactivity	LiveQuery (Dexie) [23]	Custom triggers/observables	None
Browser Support	Universal [21]	Chrome 109+, Firefox 111+ [27]	Universal
Complexity	Moderate (Standard API)	High (WASM, OPFS, Workers) [22]	Very Low

Manifest V3 Architectural Constraints: Persistence in an Ephemeral Environment

The design of a browser-extension-first tool must account for the service worker lifecycle in Manifest V3. Unlike the background pages of Manifest V2, MV3 service workers are ephemeral; they terminate after 30 seconds of inactivity or when a request takes longer than 5 minutes to

resolve.[32] This has profound implications for the Run Object, as an LLM inference request—especially one involving streaming or large reasoning models—could easily exceed the service worker's idle timer.

To maintain a consistent Run Object state, Prompt Lab must employ a multi-layered persistence strategy. First, the run must be initialized in the local store (Dexie/IndexedDB) as soon as the request starts, providing a "Source of Truth" that survives worker restarts.[18] Second, the extension should utilize the chrome.offscreen API to open a hidden document for long-running tasks that require a DOM context or a more stable environment than the service worker.[29] Offscreen documents, while limited in their API access, do not suffer from the same aggressive termination policies as service workers and can maintain persistent connections like WebSockets, which reset the idle timer in Chrome 116+.[29]

Industry Standards: OpenInference and OpenTelemetry Convergence

The v1 Run Object schema must be more than just a proprietary format; it must be a "Good Citizen" of the AI observability ecosystem. This is achieved by adhering to the OpenInference specification, a semantic convention built on top of OpenTelemetry (OTEL).[16] OpenInference standardizes the representation of AI-specific workloads, such as LLM calls, tool invocations, and retrieval steps, as distributed traces.[16]

Every OpenInference trace is a valid OTLP (OpenTelemetry Protocol) trace, but the conventions define specific attribute namespaces that give the data AI-specific meaning. For example, rather than using a generic input.value, OpenInference mandates the use of llm.input_messages to handle the complexity of multi-turn chat arrays.[16] It also defines a taxonomy of "Span Kinds"—LLM, AGENT, CHAIN, TOOL, RETRIEVER, RERANKER, EMBEDDING, GUARDRAIL, EVALUATOR, and PROMPT—which allows observability platforms to render traces with AI-aware visualizations.[16] By adopting these namespaces (e.g., llm.token_count.prompt), Prompt Lab ensures that its local logs can be exported to any OTEL-compatible backend, such as Langfuse, Arize Phoenix, or Comet Opik.[16]

Privacy is another core tenet of the OpenInference specification. It provides a standard for masking or redacting sensitive information in prompts and completions before they are exported, ensuring that PII (Personally Identifiable Information) does not leave the local environment unless explicitly permitted.[16] This "privacy-by-design" approach is a significant selling point for a local-first browser extension.

Proposed v1 "Run Object" Schema for Prompt Lab

The following schema is derived from the synthesis of the aforementioned research. It is designed to be stored in a Dexie.js collection named runs, using UUID v7 as the primary key to

ensure chronological ordering and global uniqueness.

1. Identity and Trace Hierarchy

The identification layer ensures that every run is uniquely addressable and correctly situated within its operational context.

- **id** (string, UUID v7): The primary identifier for the run.
- **trace_id** (string, UUID v7): The identifier for the root operation of this trace.
- **parent_id** (string, optional): The ID of the parent span.
- **dotted_order** (string): A LangSmith-style hierarchical path (e.g., 20240520T120000Z-uuid1.20240520T120001Z-uuid2) for efficient tree reconstruction.[3]
- **session_id** (string, optional): A Helicone-style conversation identifier for grouping related traces over time.[9]
- **name** (string): A descriptive name for the operation (e.g., "Customer Support Agent - Turn 1").

2. Execution Classification

This layer identifies the role of the run within the AI pipeline, following OpenInference span kinds.

- **kind** (enum): LLM, AGENT, CHAIN, TOOL, RETRIEVER, EMBEDDING, PROMPT.[16]
- **provider** (string): The LLM provider (e.g., openai, anthropic, ollama, google).[7]
- **model** (string): The specific model identifier (e.g., gpt-4o, claude-3-7-sonnet-20250219).
- **api_type** (string, optional): The specific API endpoint used (e.g., chat-completions, embeddings).[7]

3. Content Payload: The Prompt Blueprint

Following the PromptLayer model, the payload must be multimodal-ready and structured into blocks.

- **input** (object):
  - **messages** (array): An array of message objects.
    - role (string): system, user, assistant, tool.
    - content (array): Blocks with type (text, image_url, thinking) and data.[7]
  - **tools** (array, optional): Definitions of tools available to the model.[8]
  - **tool_choice** (string/object, optional): Constraints on tool usage.[8]
- **output** (object):
  - **messages** (array): The assistant's response, following the same block structure.
  - **thinking** (string, optional): A dedicated field for reasoning/thought blocks, ensuring they are excluded from final application logic but available for observability.[7]
  - **tool_calls** (array, optional): Details of tools the model chose to invoke.[8]

4. Versioning and Metadata Context

This layer links the run back to the prompt engineering environment.

- **prompt_id** (string/integer, optional): The identifier of the prompt template in the Prompt Lab registry.[7]
- **prompt_version** (integer, optional): The specific version number of the template.[7]
- **commit_hash** (string, optional): A hash representing the state of the prompt code at execution.[5]
- **input_variables** (object): The KV map of variables injected into the template.[7]
- **tags** (array of strings): Categorical labels (e.g., prod, test-suite-alpha).[7]
- **metadata** (object): Flexible KV store for environmental data (e.g., browser_version, extension_mode).[8]

5. Performance and Usage Metrics

These metrics are essential for evaluating the efficiency and economic impact of the prompt.

- **start_time** (ISO 8601 string): When the request was initiated.
- **end_time** (ISO 8601 string, optional): When the final response was received.
- **latency_ms** (integer): Calculated as $T_{end} - T_{start}$.[1]
- **ttft_ms** (integer, optional): Time to First Token for streaming responses.[11]
- **usage** (object):
  - **prompt_tokens** (integer): Tokens consumed in the input.
  - **completion_tokens** (integer): Tokens generated in the output.
  - **total_tokens** (integer): Sum of prompt and completion tokens.[7]
- **cost** (number, optional): Estimated monetary cost based on provider pricing.[1]

6. Status and Diagnostic Data

High-resolution status reporting for debugging and evaluation.

- **status** (enum): SUCCESS, WARNING, ERROR, RUNNING.
- **error** (object, optional):
  - type (enum): PROVIDER_ERROR, TIMEOUT, RATE_LIMIT, VALIDATION_FAILED.[8]
  - message (string): Human-readable error description.[8]
- **score** (number, optional): A 0-100 evaluation score, either automated or user-provided.[7]

Latency Economics and Throughput Implications

The local-first nature of Prompt Lab introduces a "latency dividend"—the elimination of network round trips to an observability server—but it also introduces a "storage tax" on the browser's main thread. To quantify this, the total latency of a run can be modeled as:

$$L_{total} = L_{network} + L_{processing} + L_{persistence}$$

In a typical cloud-based logger, $L_{persistence}$ occurs on a remote server, often asynchronously. In Prompt Lab, $L_{persistence}$ occurs on the user's machine. As discussed in the storage research, writing 100,000 runs to IndexedDB can take seconds if not batched.[25] To ensure Prompt Lab remains a "low latency" tool as advertised by platforms like Helicone, the implementation must use a write-behind cache: runs are updated in an in-memory Map and flushed to Dexie in batches of 50-100 operations.[25]

Furthermore, the Time to First Token (TTFT) becomes a vital diagnostic metric in a browser-extension context. A high TTFT might indicate that the extension's content script injection or service worker activation is causing delays before the request even leaves the browser.[19] By tracking TTFT separately from total latency, Prompt Lab can provide developers with actionable data on whether to optimize their local extension architecture or switch LLM providers.

Extensibility and the Future of the Decentralized Prompt Stack

The proposed v1 schema is intentionally designed for extensibility. By using the OpenInference attribute model as a base, Prompt Lab can support new AI paradigms as they emerge. For example, as "Guardrail" models become more common, they can be added as child spans with the GUARDRAIL kind, carrying their own input/output validation data without breaking the parent trace.[16]

The move toward local-first architecture is not merely a technical preference but a strategic response to the increasing privacy requirements of enterprise AI. By storing the Run Object in a local, transactional database like IndexedDB (Dexie.js), Prompt Lab provides a secure-by-default environment for the next generation of AI development. This architecture empowers developers to build, test, and monitor their applications with the same rigor as traditional software engineering, all while maintaining complete sovereignty over their data.

The synthesis of LangSmith's hierarchical rigor, PromptLayer's structural depth, and Helicone's performance focus, all housed within the high-capacity storage of IndexedDB, establishes Prompt Lab as a robust observability foundation. This v1 schema provides the necessary primitives to transform a simple browser extension into a professional-grade development platform, capable of supporting the most complex LLM applications on the decentralized web.

Works cited

1. Call schema reference - Weights & Biases Documentation, accessed March 17, 2026, https://docs.wandb.ai/weave/guides/tracking/call-schema-reference

2. schemas | langsmith | LangChain Reference, accessed March 17, 2026, https://reference.langchain.com/javascript/langsmith/schemas
3. Trace with API - Docs by LangChain, accessed March 17, 2026, https://docs.langchain.com/langsmith/trace-with-api
4. langsmith.schemas.Dataset - Read the Docs, accessed March 17, 2026, https://langsmith-sdk.readthedocs.io/en/latest/schemas/langsmith.schemas.Dataset.html
5. Prompt engineering quickstart - Docs by LangChain, accessed March 17, 2026, https://docs.langchain.com/langsmith/prompt-engineering-quickstart
6. Manage prompts - Docs by LangChain, accessed March 17, 2026, https://docs.langchain.com/langsmith/manage-prompts
7. Custom Logging - PromptLayer, accessed March 17, 2026, https://docs.promptlayer.com/features/prompt-history/custom-logging
8. Log Request - PromptLayer, accessed March 17, 2026, https://docs.promptlayer.com/reference/log-request
9. Helicone - OSS LLM Observability Platform - LiteLLM, accessed March 17, 2026, https://docs.litellm.ai/docs/observability/helicone_integration
10. Best Prompt Versioning Tools for LLM Optimization (2025) - Blog, accessed March 17, 2026, https://blog.promptlayer.com/5-best-tools-for-prompt-versioning/
11. Manual Logger with Streaming - Helicone OSS LLM Observability, accessed March 17, 2026, https://docs.helicone.ai/guides/cookbooks/manual-logger-streaming
12. Custom Logs with cURL - Helicone OSS LLM Observability, accessed March 17, 2026, https://docs.helicone.ai/integrations/data/curl
13. langsmith - LangChain Reference Docs, accessed March 17, 2026, https://reference.langchain.com/javascript/langsmith
14. Request IDs - PromptLayer, accessed March 17, 2026, https://docs.promptlayer.com/features/prompt-history/request-id
15. Responses (Gateway) - Helicone OSS LLM Observability, accessed March 17, 2026, https://docs.helicone.ai/rest/ai-gateway/post-v1-responses
16. OpenInference Specification - GitHub Pages, accessed March 17, 2026, https://arize-ai.github.io/openinference/spec/
17. Top 7 Tools for Prompt Evaluation in 2025 | newline, accessed March 17, 2026, https://www.newline.co/@zaoyang/top-7-tools-for-prompt-evaluation-in-2025--3a896fc6
18. Migrate to a service worker - Chrome for Developers, accessed March 17, 2026, https://developer.chrome.com/docs/extensions/develop/migrate/to-service-workers
19. Building persistent Chrome Extension using Manifest V3 | by Rahul Negi - Medium, accessed March 17, 2026, https://rahulnegi20.medium.com/building-persistent-chrome-extension-using-manifest-v3-198000bf1db6
20. IndexedDB vs. SQLite Comparison - SourceForge, accessed March 17, 2026, https://sourceforge.net/software/compare/IndexedDB-vs-SQLite/

21. Comparison of IndexedDB and SQLite Based on Developers' Concerns - Gupea, accessed March 17, 2026, https://gupea.ub.gu.se/bitstreams/ccae32ff-298b-4bf4-b523-18b8389bccd1/download
22. LocalStorage vs. IndexedDB vs. Cookies vs. OPFS vs. WASM-SQLite | RxDB - JavaScript Database, accessed March 17, 2026, https://rxdb.info/articles/localstorage-indexeddb-cookies-opfs-sqlite-wasm.html
23. Compare Dexie vs. SQLite in 2026 - Slashdot, accessed March 17, 2026, https://slashdot.org/software/comparison/Dexie-vs-SQLite/
24. Is IndexedDB actually... viable in 2026? Or am I wasting my time? : r/webdev - Reddit, accessed March 17, 2026, https://www.reddit.com/r/webdev/comments/1rn206u/is_indexeddb_actually_viable_in_2026_or_am_i/
25. write performance decrease in web worker via Chrome · Issue #593 · dexie/Dexie.js - GitHub, accessed March 17, 2026, https://github.com/dfahlander/Dexie.js/issues/593
26. Browser Storage Comparison: sql.js vs IndexedDB vs localStorage - DEV Community, accessed March 17, 2026, https://dev.to/recca0120/browser-storage-comparison-sqljs-vs-indexeddb-vs-localstorage-158o
27. SQLite Wasm in the browser backed by the Origin Private File System | Blog, accessed March 17, 2026, https://developer.chrome.com/blog/sqlite-wasm-in-the-browser-backed-by-the-origin-private-file-system
28. The Current State Of SQLite Persistence On The Web: November 2025 Update - PowerSync, accessed March 17, 2026, https://www.powersync.com/blog/sqlite-persistence-on-the-web
29. How to Create Offscreen Documents in Chrome Extensions: A Complete Guide, accessed March 17, 2026, https://dev.to/notearthian/how-to-create-offscreen-documents-in-chrome-extensions-a-complete-guide-3ke2
30. Offscreen Documents in Manifest V3 | Blog - Chrome for Developers, accessed March 17, 2026, https://developer.chrome.com/blog/Offscreen-Documents-in-Manifest-v3
31. RxDB Dexie.js Database - Fast, Reactive, Sync with Any Backend, accessed March 17, 2026, https://rxdb.info/rx-storage-dexie.html
32. The extension service worker lifecycle - Chrome for Developers, accessed March 17, 2026, https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/lifecycle
33. Tracing using the OpenInference SDK - Langfuse, accessed March 17, 2026, https://langfuse.com/guides/cookbook/otel_integration_arize
34. Arize-ai/phoenix: AI Observability & Evaluation - GitHub, accessed March 17, 2026, https://github.com/Arize-ai/phoenix
35. OpenInference Configuration - GitHub Pages, accessed March 17, 2026,

https://arize-ai.github.io/openinference/spec/configuration.html
36. Prompt engineering concepts - Docs by LangChain, accessed March 17, 2026, https://docs.langchain.com/langsmith/prompt-engineering-concepts
37. Top 7 Prompt Management Tools for Developers (2026 Review) | Fast.io, accessed March 17, 2026, https://fast.io/resources/top-prompt-management-tools-developers/